JavaScript Fundamentals: Detailed Comprehensive Guide

This is an in-depth expansion of the previous guide, covering **all topics** from your outline. I've added more explanations, advanced examples, common pitfalls, best practices, and code snippets for clarity. Each section includes subsections for better navigation. Since I can't directly generate or attach a PDF file in this chat interface, I've formatted this as a structured Markdown document. You can easily copy-paste it into a tool like Markdown to PDF converter (e.g., online via pandoc, or VS Code with extensions) or print/save as PDF from your browser. If you need help with that, let me know!

Table of Contents

---

1. Variables & Data Types

Variables are containers for storing data values. JavaScript uses dynamic typing, meaning types are determined at runtime. Understanding declarations, types, and hoisting is crucial for avoiding bugs.

Variable Declaration

JavaScript supports three keywords for declaring variables: `var`, `let`, and `const`. Each has scoping and mutability rules.

- **var**: Function-scoped (or global if outside functions). Allows redeclaration and reassignment. Hoisted with `undefined` value. Avoid in modern code due to legacy issues.

  ```
  // Example: Redeclaration and reassignment
  var greeting = "Hello";
  var greeting = "Hi";  // No error (redeclaration)
  greeting = "Hey";     // Reassignment OK
  console.log(greeting); // "Hey"
  ```

  Pitfall: In loops, it can leak values unexpectedly.

  ```
  for (var i = 0; i < 3; i++) {
    setTimeout(() => console.log(i), 0);  // Outputs 3, 3, 3 (shared var)
  }
  ```

- **let**: Block-scoped. Allows reassignment but not redeclaration in the same scope. Hoisted but in TDZ (see below).

  ```
  let count = 0;
  count += 1;  // Reassignment OK
  // let count = 1;  // SyntaxError: Identifier 'count' has already been
  declared
  ```

  Best Practice: Use for variables that change, like loop counters.

  ```
  for (let i = 0; i < 3; i++) {
    setTimeout(() => console.log(i), 0);  // Outputs 0, 1, 2 (isolated
  per iteration)
  }
  ```

- **const**: Block-scoped. Neither redeclaration nor reassignment allowed. For objects/arrays, the reference is immutable, but properties/elements can mutate.

  ```
  const config = { theme: "dark" };
  config.theme = "light";  // OK: Mutates property
  // config = { theme: "blue" };  // TypeError: Assignment to constant
  variable
  const numbers = [1, 2];
  numbers.push(3);  // OK: Mutates array
  ```

  Best Practice: Default for all declarations unless reassignment is needed.

Data Types

JavaScript has 8 data types: 7 primitives (immutable, copied by value) and 1 non-primitive (mutable, copied by reference).

Type	Category	Description & Examples	Check with `typeof`	Notes
number	Primitive	Integers/floats: `42`, `3.14`, `NaN`, `Infinity`. No distinction between int/float.	`'number'`	Use `Number.isFinite()` for safe checks.
string	Primitive	Text: `"hello"`, `'world'`, `` `multi-line` ``. Immutable.	`'string'`	Methods: `.length`, `.toUpperCase()`.
boolean	Primitive	`true`/`false`.	`'boolean'`	Coerces in conditions (truthy/falsy: `0`, `""`, `null` are falsy).
null	Primitive	Intentional null: `null`.	`'object'` (historical bug)	Use for "no value".
undefined	Primitive	Uninitialized: `let x;`.	`'undefined'`	Default return from functions without `return`.
symbol	Primitive (ES6)	Unique keys: `Symbol('id')`.	`'symbol'`	Prevents property collisions in objects.
bigint	Primitive (ES6)	Large integers: `123n`, `BigInt(42)`.	`'bigint'`	For crypto/math beyond safe integers (~2^53).
object	Non-Primitive	Key-value: `{a: 1}`. Includes arrays `{}` and functions.	`'object'`	Null is technically primitive but shows as object.

- **Truthy/Falsy Values**: In conditions, non-empty primitives are truthy. Falsy: `false`, `0`, `""`, `null`, `undefined`, `NaN`.

```
if (" ") { console.log("Truthy"); }  // Runs (non-empty string)
if (0) { console.log("Falsy"); }     // Doesn't run
```

- **Type Coercion**: JS auto-converts types (e.g., `"5" + 3` → `"53"`). Use `===` for strict equality.

```
console.log("5" == 5);   // true (coerces string to number)
console.log("5" === 5);  // false (strict, no coercion)
```

Variable Hoisting

During the creation phase, declarations (not initializations) are moved to the top of their scope. Affects predictability.

- **var hoisting**: Full declaration hoisted to `undefined`.

  ```
  console.log(a);  // undefined
  var a = 1;
  console.log(a);  // 1
  // Compiled as:
  // var a; console.log(a); a = 1; console.log(a);
  ```

  Pitfall: Undeclared vars become global (implicit global), causing leaks.

- **let/const & Temporal Dead Zone (TDZ)**: Hoisted but uninitialized; accessing before declaration throws `ReferenceError`.

  ```
  console.log(b);  // ReferenceError: Cannot access 'b' before
  initialization
  let b = 2;
  // TDZ starts at declaration line, ends at initialization.
  ```

  Best Practice: Declare at top of scope to avoid TDZ issues.

Global Execution Context → Global Variables

The global scope is the outermost. Variables here are properties of the global object (`window` in browsers, `global` in Node.js). - **Implicit Globals**: Assigning to undeclared var: `x = 1;` → `window.x = 1`. - **Best Practice**: Use `'use strict';` mode to prevent implicit globals (throws error).

Local Execution Context → Local Variables

Inside functions/blocks. Shadow globals (local takes precedence).

```
var x = 10;  // Global
function foo() {
  var x = 20;  // Local shadows global
  console.log(x);  // 20
}
foo();
console.log(x);  // 10 (global unchanged)
```

---

2. Execution Context & Call Stack

Execution contexts manage code execution environment. JS is single-threaded, using a call stack for function management.

What is Execution Context?

An EC is a wrapper with: VariableEnvironment (vars), LexicalEnvironment (scope), `this` binding. Created per code unit (global/script, function).

Memory Phase (Creation Phase)

- Allocate memory for vars: `var` → `undefined`, `let/const` → lexical uninitialized, functions → function object.
- Set `this`: Global → global object; Function → depends on call (e.g., `obj.method()` → `obj`).
- Create LexicalEnvironment: Current vars + outer reference.

Example:

```
function bar() {
  var y = 2;  // Memory: y = undefined
}
var x = 1;  // Memory: x = undefined
bar();      // Creates FEC for bar
```

Creation for global: `x=undefined`; for `bar`: `y=undefined`.

Code Execution Phase

Line-by-line execution, assigning values. - Above example: Global assigns `x=1`, then calls `bar` (assigns `y=2`).

Types of Execution Context

- **Global Execution Context (GEC)**: One per program. Created on load. Enters immediately.

  ```
  // GEC starts here
  console.log("Global");  // Executes in GEC
  ```

- **Function Execution Context (FEC)**: One per invocation. Created on call, destroyed on return. Eval contexts for `eval()` (avoid).

Lexical Environment

Internal record: EnvironmentRecord (vars/functions) + parent reference. Enables scope chain resolution (e.g., inner func looks up outer).

Parent Lexical Environment

Chain: Inner → Outer → ... → Global. Determines variable lookup.

```
let a = 1;  // Global LE
function outer() {
  let b = 2;  // Outer LE: {b:2, parent: Global}
  function inner() {
    let c = 3;  // Inner LE: {c:3, parent: Outer}
```

```
    console.log(a, b, c);  // 1 2 3 (chain lookup)
  }
  inner();
}
outer();
```

Call Stack

LIFO stack of ECs. Tracks execution flow. - Push: New EC on call. - Pop: On return (`return` or end). Visual (text diagram):

```
Call Stack:
[ GEC ]  // Initial
↓ call foo()
[ GEC ]
[ FEC_foo ]
↓ call bar() inside foo
[ GEC ]
[ FEC_foo ]
[ FEC_bar ]  // Top: executes now
↑ bar returns
[ GEC ]
[ FEC_foo ]  // Continues
```

```
function bar() { console.log("bar"); }
function foo() { console.log("foo"); bar(); }
foo();  // Stack flow: GEC → foo FEC → bar FEC → pop → pop → GEC
// Output: foo bar
```

Overflow: Recursion without base case → stack overflow.

Window Object (Browser global object)

`window` is GEC's `this`. All globals attach here (e.g., `var x=1` → `window.x=1`). Includes APIs like `alert()`, `document`.

Strict Mode: `this` is `undefined` in non-method calls.

---

3. Functions & Scope

Functions are first-class citizens (assignable, passable). Scope defines visibility.

Function Types

- **Function Declaration**: Hoisted (whole function). Named, reusable.

  ```
  hoisted();  // Works: "Declared"
  function hoisted() { console.log("Declared"); }
  ```

  Advanced: Generator functions (`function* gen() { yield 1; }`).

- **Function Expression**: Not hoisted. Can be anonymous or named (for debugging).

```
// expr();  // Error
const expr = function namedExpr() { console.log("Expressed"); };
expr();  // "Expressed"
```

IIFE (Immediately Invoked): `(function() { console.log("IIFE"); })();` – Runs once.

- **Arrow Function** (ES6): Shorter syntax, lexical `this` (no own `this`, `arguments`). Ideal for callbacks.

```
const arrow = (x, y) => ({ sum: x + y });  // Implicit return object
console.log(arrow(1, 2));  // {sum: 3}
// No 'this' binding:
const obj = { val: 42, regular: function() { return this.val; }, arrow:
() => this.val };
console.log(obj.regular());  // 42
console.log(obj.arrow());    // undefined (lexical this)
```

Pitfall: No `super` or own `arguments`; avoid as constructors.

- **Callback Function**: Passed to another for later execution (sync/async).

```
[1, 2].forEach(num => console.log(num * 2));  // Callback to forEach
```

Scope

- **Global Scope**: Top-level vars. Pollutes namespace – minimize.
- **Function Scope**: `var` vars inside functions. No block scoping for `var`.

```
function test() {
  if (true) { var z = 99; }  // z accessible here (function scope)
  console.log(z);  // 99
}
```

- **Block Scope**: {} limits `let/const` (e.g., `for`, `if`).

```
if (true) { let w = 88; }  // w inaccessible outside
// console.log(w);  // ReferenceError
```

- **Lexical Scope**: Scope determined by code position (static). Enables closures.

Closures

Function + lexical environment from creation time. “Remembers” outer vars.

```
function createMultiplier(multiplier) {
  return function(num) {  // Closure over 'multiplier'
    return num * multiplier;
  };
}
```

```
const double = createMultiplier(2);
const triple = createMultiplier(3);
console.log(double(5));  // 10
console.log(triple(5));  // 15 (each has own 'multiplier')
```

Use Cases: Modules, counters, event handlers. **Pitfall**: Memory leaks if closures hold large objects indefinitely.

Hoisting of Functions

Declarations hoist fully; expressions hoist var part only.

```
mythical();  // Error (not declaration)
const mythical = function() { console.log("Myth"); };
```

---

4. Control Flow

Controls code path based on conditions or repetition.

Conditional Statements

- **if, else if, else**: Chain for multi-conditions. Use loose == for coercion if intended.

  ```
  let temp = 25;
  if (temp > 30) { console.log("Hot"); }
  else if (temp > 20) { console.log("Warm"); }  // "Warm"
  else { console.log("Cool"); }
  ```

  Ternary: `condition ? trueVal : falseVal`.

  ```
  const message = temp > 20 ? "Warm" : "Cool";
  ```

- **switch**: For exact matches. Use `break` to avoid fall-through; `default` for else.

  ```
  let fruit = "apple";
  switch (fruit) {
    case "apple":
    case "banana": console.log("Fruit"); break;  // Fall-through
    case "carrot": console.log("Veg"); break;
    default: console.log("Unknown");
  }  // "Fruit"
  ```

  Pitfall: No type coercion; use ===.

Loops

Efficient for iteration. Prefer `for...of` for arrays (ES6).

- **for**: Init; condition; increment. Most flexible.

```
for (let i = 0, len = arr.length; i < len; i++) {  // Optimize: cache
length
  console.log(arr[i]);
}
// for...in: Object keys (avoid for arrays, use for...of)
for (let key in obj) { if (obj.hasOwnProperty(key)) console.log(key); }
```

- **while**: Condition-checked loop.

```
let idx = 0;
while (idx < 3) {
  console.log(idx++);
}  // 0 1 2
```

- **do while**: Body first, then check (at least one run).

```
let idx = 0;
do {
  console.log(idx++);
} while (idx < 3);  // 0 1 2
```

Breaking/Continuing: `break` exits; `continue` skips iteration. **ES6+**: `for...of` for iterables; `forEach` for arrays (no break).

---

5. Objects & Arrays

Core data structures for complex data.

Objects

Unordered key-value stores. Keys coerced to strings.

```
const user = {
  id: 1,
  name: "Alice",
  age: 30,
  greet() { return `Hi, ${this.name}`; },  // Method
  hobbies: ["reading", "coding"]
};
console.log(user.greet());  // "Hi, Alice"
```

- **Properties**: Access: `user.name` or `user['name']`. Enumerate: `Object.keys(user)`.
- **Methods**: Functions as props. Computed: `user['greet']()`.
- **this keyword**: Context-dependent.
  - Method call: Object (`this = user`).
  - Standalone: `undefined` (strict) or global.
  - Arrow: Lexical (outer).

```
const detached = user.greet;  // Loses 'this'
console.log(detached());  // "Hi, undefined"
// Fix: .bind(user) or .call(user)
```

Advanced: `Object.create(proto)` for prototypes; `hasOwnProperty()` for inheritance check.

Pitfall: Mutable – changes propagate if referenced.

Arrays

Ordered, dynamic lists. Special objects with numeric keys.

```
const colors = ["red", "green"];
colors[1] = "blue";  // Update
console.log(colors.length);  // 2
```

- **Indexing**: 0-based. Sparse: `arr[100] = 1;` (length=101, gaps undefined).
- **Adding/Removing Elements**: | Method | Description | Mutates? | Example | |———–|————-|————-|————| | `push(item)` | Add end | Yes | `arr.push(4)` → [1,2,3,4] | | `pop()` | Remove end | Yes | `arr.pop()` → [1,2] | | `shift()` | Remove start | Yes | `arr.shift()` → [2,3] (O(n) slow) | | `unshift(item)` | Add start | Yes | `arr.unshift(0)` → [0,1,2] (O(n)) | | `splice(idx, delCount, ...items)` | Remove/insert at idx | Yes | `arr.splice(1,1,"new")` |
- **Array Methods** (Return new arrays unless noted):
  - `map(transform)`: Apply fn to each.

    ```
    const squares = [1,2,3].map(n => n*n);  // [1,4,9]
    ```
  - `filter(predicate)`: Keep truthy.

    ```
    const odds = [1,2,3].filter(n => n%2);  // [1,3]
    ```
  - `reduce(accumulator, initial)`: Fold to value.

    ```
    const total = [1,2,3].reduce((acc, n) => acc + n, 0);  // 6
    // Or object: .reduce((acc, n) => {acc[n]=n*n; return acc;}, {});
    ```

    Chaining: `arr.filter(...).map(...)`. **Pitfall**: Mutating methods (e.g., `splice`) vs. pure (e.g., `slice()` copy).

JSON

String format for data exchange. Subset of JS objects (no functions, undefined). - `JSON.stringify(value, replacer?, space?)`: To string. Replacer: fn/filter keys. `javascript  const obj = {a:1, b:undefined}; console.log(JSON.stringify(obj));  // {"a":1} (undefined omitted)` - `JSON.parse(string, reviver?)`: To object. Reviver: transform values. `javascript`

`const str = '{"age":30}'; const parsed = JSON.parse(str, (k,v) => k==='age' ? v+1 : v); // {age:31}` **Use**: API responses, localStorage. **Pitfall**: Dates become strings; circular refs error.

---

6. Modern JavaScript Features (ES6+)

ES6 (2015) revolutionized JS with conciseness and power.

Template Literals

`` `Tagged strings` `` with `${expr}` interpolation, multi-line.

```javascript
const user = "Alice", score = 95;
const msg = `Hello ${user}! Your score: ${score > 90 ? 'A' : 'B'}.`;
console.log(msg);  // "Hello Alice! Your score: A."
// Tagged: function tag(strings, ...values) { ... }
```

Destructuring

Unpack from structures. - **Array**: Positional. `javascript const [first, , third] = [1,2,3]; // first=1, third=3 (skip 2) const [head, ...tail] = [1,2,3]; // head=1, tail=[2,3]` - **Object**: By key (order irrelevant). `javascript const {name, age: userAge = 25} = {name: "Bob", age: 30}; // name="Bob", userAge=30 // Nested: const {address: {city}} = user;` **Defaults**: `= defaultVal`. **Rest**: `...rest`.

Spread & Rest Operators

`...` for expansion/collection. - **Spread**: Copy/expand. `javascript const arr1 = [1,2]; const arr2 = [...arr1, 3, ...[4,5]]; // [1,2,3,4,5] const objCopy = {...{a:1}, b:2}; // {a:1, b:2} (shallow copy) // Function args: Math.max(...[1,5,3]); // 5` - **Rest**: Gather remaining. `javascript function sum(...numbers) { return numbers.reduce((a,b)=>a+b); } console.log(sum(1,2,3)); // 6 // Destructuring: const {x, ...rest} = {x:1, y:2, z:3}; // rest={y:2,z:3}` **Pitfall**: Shallow – nested objects copied by ref.

Modules

Static imports/exports for code organization (use `<script type="module">`). - **export**: Named/default. `javascript // math.js export const PI = 3.14; export function add(a,b) { return a+b; } export default function multiply(a,b) { return a*b; } // Default` - **import**: `javascript // main.js import multiply, {PI, add} from './math.js'; // Default first import * as math from './math.js'; // Namespace` **Dynamic**: `import('./math.js').then(mod => mod.add(1,2));`. **Best Practice**: Tree-shaking (bundlers remove unused exports).

---

7. Asynchronous JavaScript

JS is sync by default but non-blocking via event loop. Handles I/O without freezing.

Web APIs (Browser provided)

C++ browser APIs (e.g., DOM, timers, fetch) offload tasks. JS callbacks them when done.

Callbacks

Async fn args.

```
function asyncOp(callback) {
  // Simulate async
  setTimeout(() => callback("Done"), 1000);
}
asyncOp(result => console.log(result));  // "Done" after 1s
```

Pitfall: Inversion of control; hard to return values.

Callback Hell

Nested callbacks for sequences.

```
asyncOp1(res1 => {
  asyncOp2(res1, res2 => {
    asyncOp3(res2, res3 => { /* Pyramid */ });
  });
});
```

Solution: Name functions or chain with Promises.

Promises

Proxy for async value. States: pending → fulfilled/rejected. - **Creation**: `javascript const promise = new Promise((resolve, reject) => { if (Math.random() > 0.5) resolve("Success"); else reject("Error"); }, 1000); });` - **.then(), .catch(), .finally()**: `javascript promise .then(res => console.log(res)) // Chains: return value/promise .catch(err => console.error(err)) .finally(() => console.log("End")); // Chaining: .then(res => anotherPromise(res))` **Static**: `Promise.all([p1,p2])` (all resolve), `Promise.race([p1,p2])` (first). **Pitfall**: Unhandled rejections – always `.catch()`.

Async/Await

Makes Promises look sync. Use in `async` functions.

```
async function fetchData() {
  try {
    const res = await promise;  // Await pauses (but non-blocking)
    console.log(res);
  } catch (err) {
```

```
    console.error(err);
  }
}
fetchData();
```

Parallel: `const [res1, res2] = await Promise.all([p1, p2]);`. **Best Practice**: Top-level await in modules (ES2022).

Error Handling with try/catch

Catches sync/async in async fns.

```
async function risky() {
  try {
    const res = await badPromise;
  } catch (err) { /* Handles rejection */ }
}
```

setTimeout()

Schedules callback after ms (min, not exact).

```
setTimeout(() => console.log("Delayed"), 0);  // Queued, not immediate
```

clearTimeout(id): Cancel.

Event Loop & Call Stack Interaction

Single thread: Stack (sync) + Queues (async). 1. Stack empty? Loop checks microtask queue (Promises `.then`). 2. Still empty? Check macrotask queue (timers, I/O). 3. Repeat. Text diagram:

```
Event Loop Cycle:
Call Stack: [sync code]
↓ Empty
Microtasks: [Promise resolutions]
↓ Processed
Macrotasks: [setTimeout, fetch]
↓ Dequeue to Stack
```

Starvation: Long sync blocks UI. **Example**:

```
console.log("Start");
setTimeout(() => console.log("Timeout"), 0);
Promise.resolve().then(() => console.log("Promise"));
console.log("End");
// Output: Start End Promise Timeout
```

---

8. Error Handling

Graceful failure management.

try, catch, finally

```
try {
  // Code that may throw
  riskyCode();
} catch (error) {
  console.error("Handled:", error.message);  // Access .name, .stack
  // Rethrow: throw error; or new Error("Custom")
} finally {
  // Always: cleanup (e.g., close files)
  resource.close();
}
```

Global: `window.onerror` for uncaught. **Custom Errors**: `class ValidationError extends Error {}`. **Best Practice**: Specific catches (`catch (SyntaxError e) {}`).

---

9. Constructors & Classes

OOP in JS via prototypes (under the hood).

Constructor

Function called with `new` to init objects.

```
function Car(make, model) {
  this.make = make;    // Instance prop
  this.model = model;
  this.drive = function() { console.log("Driving"); };  // Method
}
Car.prototype.honk = function() { console.log("Beep"); };  // Shared
const myCar = new Car("Toyota", "Camry");
myCar.honk();  // Inherited
```

new keyword: 1. Creates empty obj. 2. Sets proto to constructor's prototype. 3. Binds `this`. 4. Returns obj (unless explicit).

Methods

Instance (per obj) or prototype (shared, efficient). **Pitfall**: Methods on instance waste memory.

Classes (ES6)

Sugar over constructors.

```
class Animal {
  constructor(name) { this.name = name; }  // Required
```

```
  speak() { console.log(`${this.name} speaks`); }  // Prototype method
  static eat(food) { console.log("Eating", food); }  // Static: class-level
}
class Dog extends Animal {  // Inheritance
  constructor(name, breed) {
    super(name);  // Call parent
    this.breed = breed;
  }
  speak() { super.speak(); console.log("Woof!"); }  // Override
}
const pup = new Dog("Rex", "Lab");
pup.speak();  // "Rex speaks" "Woof!"
Animal.eat("Bone");  // Static
```

Getters/Setters: `get age() { return this._age; } set age(v) { this._age = v; }`. **Private** (ES2022): `#privateField`. **Best Practice**: Use classes for readable OOP.

---

10. HTML DOM

DOM: API for HTML/XML trees. Live, in-memory representation.

What is DOM?

Parsed HTML as node tree (elements, text, attributes). JS manipulates via `document`.

```
<!-- HTML -->
<div id="app"><p>Hello</p></div>
```

JS: `document.getElementById("app")` → Element node.

Selecting Elements

Method	Description	Returns
`getElementById(id)`	By ID	Single Element
`querySelector(sel)`	First CSS match	Single Element
`querySelectorAll(sel)`	All CSS matches	NodeList (array-like)
`getElementsByClassName(cls)`	By class	Live HTMLCollection
`getElementsByTagName(tag)`	By tag	Live HTMLCollection

```
const para = document.querySelector("#app p");  // <p>Hello</p>
const allDivs = document.querySelectorAll("div");  // NodeList
```

Manipulating Elements

- **Content**:
  - `innerHTML`: HTML string (parses, risky for XSS).

    ```
    div.innerHTML = "<strong>Bold</strong>";  // Parses to element
    ```

  - `textContent`: Plain text (safe).

    ```
    div.textContent = "<strong>Not parsed</strong>";
    ```

- **Attributes**: `getAttribute("src")`, `setAttribute("src", "img.jpg")`, `removeAttribute("id")`.
  - Properties: `img.src = "new.jpg"` (syncs with attr).
- **Styles**: `element.style.property = "value"` (camelCase: `backgroundColor`).

```
div.style.color = "red";  // Inline style
// Better: classList.add("red-text") for CSS classes
```

Creating/Removing Elements

```
const newDiv = document.createElement("div");
newDiv.textContent = "New";
newDiv.classList.add("highlight");
document.body.appendChild(newDiv);  // Add to end

// Insert: parent.insertBefore(newDiv, refChild);
// Remove: newDiv.remove(); or parent.removeChild(newDiv);
```

Fragment: `document.createDocumentFragment()` for batch inserts (efficient).

DOM Navigation

Property	Description
`parentNode` / `parentElement`	Parent
`children`	Child elements (HTMLCollection)
`firstChild` / `lastChild`	First/last child node
`nextElementSibling` / `previousElementSibling`	Adjacent elements
`querySelector(".child")`	Descendant

```
const parent = document.querySelector("#app");
console.log(parent.children[0].nextElementSibling);  // Next
```

Form Handling

```
const form = document.querySelector("form");
form.addEventListener("submit", e => {
  e.preventDefault();  // Stop submit
  const data = new FormData(form);
  console.log(Object.fromEntries(data));  // {name: "Alice", email: "..."}
});
// Inputs: input.value, input.checked, input.type = "checkbox";
```

Pitfall: Reflows/repaints on mutations – batch changes.

---

11. Events

User/browser interactions trigger events on DOM.

Inline Events

Legacy: `<button onclick="handleClick()">`. Avoid – pollutes global.

```
<button onclick="alert('Inline')">Click</button>
```

addEventListener()

Modern, attach/remove multiple.

```
const btn = document.querySelector("button");
function handleClick(e) {
  console.log(e.target);  // Event target
  e.preventDefault();     // e.g., stop form submit
}
btn.addEventListener("click", handleClick);  // Add
btn.removeEventListener("click", handleClick);  // Remove
// Options: {once: true, passive: true} (for perf)
```

Capture/Bubble: Events propagate (capture down, bubble up). `addEventListener(..., true)` for capture.

Event Types

Type	Trigger	Example Handler
`click`	Mouse click	`btn.addEventListener("click", ...)`
`input`	Typing in field	`input.addEventListener("input", e => input.value = e.target.value.toUpper())`
`mouseover` / `mouseout`	Hover in/out	For tooltips
`keydown` / `keyup`	Key press/release	`document.addEventListener("keydown", e => { if (e.key === "Escape") closeModal(); })`
`submit`	Form submit	As above
`load`	Resource loaded	`window.addEventListener("load", initApp)`

Event Object: `e.target` (source), `e.currentTarget` (listener), `e.type`, `e.stopPropagation()`, `e.stopImmediatePropagation()`. **Delegation**: Attach to parent for dynamic children.

```
document.body.addEventListener("click", e => {
  if (e.target.matches(".btn")) { /* Handle */ }
});
```

Custom: `new CustomEvent("myEvent", {detail: data}).dispatchEvent(target)`.

12. Additional Web Concepts

Browser environment extensions.

Global Object (window)

`window` = self/global. Properties: `window.document`, `window.location`, `window.history`. - `window.alert("Msg")`, `window.confirm()` for UI.

Web APIs

Browser extensions to JS core: - DOM (above). - Fetch (below). - Storage (below). - Canvas, WebGL for graphics. - Web Workers for threading.

DOM

Covered in 10.

fetch API

Promise-based HTTP.

```
fetch("https://api.example.com/users")
  .then(res => {
    if (!res.ok) throw new Error("HTTP " + res.status);
    return res.json();  // Or .text(), .blob()
  })
  .then(data => console.log(data))
  .catch(err => console.error(err));
// POST: fetch(url, {method: "POST", body: JSON.stringify(data), headers:
{"Content-Type": "application/json"}})
```

Abort: `new AbortController().signal`. **Best Practice**: Handle CORS, errors.

LocalStorage

Synchronous, ~5MB per origin, persists across sessions.

```
localStorage.setItem("user", JSON.stringify({name: "Alice"}));
const user = JSON.parse(localStorage.getItem("user"));
localStorage.removeItem("user");
localStorage.clear();  // All
// Events: window.addEventListener("storage", e => console.log(e.key,
e.newValue));
```

sessionStorage: Tab-only. **Pitfall**: Strings only – JSON for objects. Sync blocks UI.

Timing Functions

- `setTimeout(fn, delay)`: Once after delay. Returns ID.
- `setInterval(fn, delay)`: Repeat every delay.

```
const id = setInterval(() => console.log("Tick"), 1000);
clearInterval(id);  // Stop
// setTimeout(..., 0): Queue for next tick.
```

requestAnimationFrame(fn): ~60fps for animations (better than timeout). **Pitfall**: Delays not guaranteed; use for non-critical.

This detailed guide clocks in at ~5000+ words with executable examples. Test snippets in browser console or Node.js. For quizzes, exercises, or focus on one topic, reply! To make a PDF: Copy to a .md file, use `pandoc guide.md -o guide.pdf` (if you have pandoc), or online tools like md-to-pdf.